20 — Accessibility and the forms

The assessment asks for forms that "suit the intended users" and for accessibility to be considered, and the unit's first learning outcome is a user-centred design that "explicitly recognises the user and is Disability Discrimination Act compliant". An audit of this submission found no accessibility work at all in the tree: no groups, no headings, no reported contrast, no WCAG check, no rationale document, and a field description on 43 of 233 fields.

This document is the work, and it is careful about what it does and does not show.

What was checked

`bpm/tools/audit_accessibility.py` reads all 36 `.form` files and reports on five rules. The output of the last run is `screenshots/v16/accessibility-report.json`.

Rule	What it asks	Before	Now
A1	does every control carry a human label rather than its variable name?	0 failures	**0**
A2	does every field explain itself?	43 of 233	**233 of 233**
A3	does a choice offer words, or is the code shown as its own label?	31 failures	**0**
A4	can an unticked box be told apart from an ignored one?	18 unexplained	**0**
A5	is a form of more than seven fields grouped?	6 ungrouped	**15 groups, 0 ungrouped**

```
$ python3 tools/audit_accessibility.py
forms 36 | fields 233 | groups 15 | described 233 | required 152
errors 0 | warnings 0
```

What changed, concretely:

- **81 option labels** across 28 forms were showing the stored code as the thing a person reads. `determine-funding-route` offered `HOSPITAL_FUNDED`, `INSURER_APPROVED`, `PATIENT_PAYS`, `EXEMPTION`; about ten fields offered plain **true** and **false** buttons. They now read "Funded by the hospital", "Approved by the insurer", "Payable by the patient", "Exempt from charge", "Yes", "No". The stored values are untouched, because the model and the workers depend on them.
- **190 field descriptions** were added, in the language of the person filling the form. A tick box now says what *not* ticking means, so an untouched box is a decision the form records rather than a gap.
- **Six long forms were grouped**: the referral pack into *Referral* and *What arrived*, the funding form into *Who is paying* and *Authorisation*, the clinic letter into *The consultation* and *Approval and distribution*, and so on. A screen reader announces a group before its fields, and a long form becomes navigable in sections.

- `consentGiven` and the other branch-driving radios now offer sentences rather than `true` / `false`, which is both an accessibility fix and the same defect as DEF-02 from the other direction.

What was not checked, and is not claimed

The audit is a **static read of the form definitions**. It cannot:

- ~~measure colour contrast, because there is no rendered page to measure~~ — **now measured; see below**;
- run a screen reader, or time a keyboard-only pass;
- confirm that form-js announces a `group` the way a native fieldset would;
- check the Tasklist chrome itself, which is Camunda's and not ours.

Those would need a headless browser driving Camunda Tasklist against a running engine. **Contrast is now done that way; the rest is not.** Saying "WCAG 2.1 AA compliant" on the strength of this audit would still be false, and it is not said anywhere in this submission.

Contrast, measured on the rendered form

`tools/audit_rendered_contrast.py` drives the real Tasklist in headless Chrome, opens a task, and walks the text nodes of the rendered form. For each one it finds the effective background by climbing to the first ancestor that paints one, then applies the WCAG 2.1 relative-luminance formula. Normal text needs 4.5:1, large text 3:1.

Measured on `SEC_Task_CheckPack` (the longest form, and the first one a case reaches), against the deployed model:

Form fields rendered	13
Text nodes measured	81
Below the WCAG floor	1
— of which live text	**0**
Lowest live ratio	**4.55:1** (the floor is 4.5:1)
Highest live ratio	18.1:1

The single sub-threshold node is the **`Complete Task` button while it is disabled**, at 1.94:1. That is recorded rather than excluded silently, because it is the honest shape of the result: a freshly opened task has required fields empty, so the button is inactive, and **WCAG 1.4.3 exempts inactive user interface components**. Counting it as a failure would be wrong; hiding it would be worse. The tool reports the two numbers separately (`failing` and `failingExcludingDisabledControls`) and exits non-zero only on the second.

The evidence is `screenshots/v16/tasklist-contrast.json` — one row per text node, with its colour, its background, its size, the floor it was held to and its ratio.

What this still does not cover. It measures one form, not all 37; the others use the same components and the same renderer, but that is an argument, not a measurement. It measures the renderer's own theme, so it says nothing about a deployment that restyles Tasklist. And it cannot see focus order, announcements or whether a keyboard-only user can finish the form.

The rationale, in one place

- **Who the users are.** Medical secretaries, booking teams, call handlers, pathway coordinators, administrative managers, consultants and nurse specialists, plus finance. Most of them work through long forms at speed, several are not clinical staff, and the person who has the least context is always the one filling in the reason.
- **What that implies.** Every field says what it is for; every choice is in words; every long form is sectioned; nothing relies on colour alone to carry meaning; and the branch-deciding questions are marked required so a form cannot be submitted with the routing answer missing.
- **What the worker side adds.** Form-level validation cannot enforce everything — `datetime` in form-js reads only `required`, and `disallowPassedDates` is a calendar constraint that never runs in the renderer's validation function. The worker-side `DateRuleValidator` is therefore the enforcement layer, not a duplicate of the form. That finding is in `docs/14-feedback-response.md` §3.
- **Accessibility needs are asked about, not assumed.** `choose-appointment-slot` has a `patientAccessibilityNeeds` field that names examples — a large print letter, a step-free route, a carer attending — and an `interpreterRequired` box. A patient's needs are recorded where the appointment is made, by the person making it.

The grouping defect, and why it is recorded here

The grouping pass that sectioned the long forms also wrote `path` onto every group — `"path": "group0"`, `"group1"`, and on `record-consent-and-treatment-request` `"consentPlan"` and `"consentDecline"`. A group's path becomes a **prefix on every child field's key** (Camunda data binding: "the group's path acts as a prefix to the key of each child field within that group"). So `supportingDocuments` was submitted as `group1.supportingDocuments`, and every worker — which reads its variables at the top level — saw nothing at all.

It went unnoticed because every automated run fills the forms over the API with flat variables, so the driver never exercised the binding the forms actually use. The symptom appeared the first time a person used Tasklist: the referral-pack validation reported the pack as **unreadable**, however carefully the form was filled, and the missing-information loop could not be left, because recording the reply changed nothing the worker could see.

Fixed in `tools/fix_form_group_paths.py`, which clears the path on all 15 groups in the 7 grouped forms and changes nothing else — same groups, same labels, same children, so the audit below still counts them. The path is documented as a property that "may be left empty", which is what it now is.

Guarded in `tools/forms_inventory.py`: a field that sits under a non-empty group path is reported as `KEY_NESTED_BY_GROUP_PATH`, so the same mistake fails the inventory instead of reaching a demonstration. Adding a path back to one group of `log-enquiry` produces three findings, one per field in that group.

The hidden required field

`record-consent-and-treatment-request.form` carried three `conditional` expressions. Two compared against a **string** — `=consentGiven = "false"` and `=consentGiven = "true"` — and one, on `treatmentPlan`, compared against a **boolean**: `=consentGiven = false`. The field is a radio, and form-js submits the string `"true"`, so that expression never held; the field stayed hidden.

A hidden required field is not validated. The form submitted happily with `treatmentPlan` empty, `treatment.record-funding-approval` recorded the funding, and the pathway then died two steps later on `TRT_Auto_ValidateRequest` with "*Required process variable 'treatmentPlan' is missing or blank*" — a long way from the field that caused it.

This is the DEF-02 defect again, in its last surviving copy: the same string/boolean confusion, in the same file, fixed in two places out of three.

Fixed by quoting the literal, so the three expressions agree. **Guarded** in `tools/forms_inventory.py`, which now reports `CONDITIONAL_TYPE_MISMATCH` when a conditional compares a field that submits a string (radio, select, textfield, textarea) against an unquoted `true` or `false`. Removing the quotes again produces one finding.

Why it survived every automated run: the driver fills only the fields a form marks **required**, and completes the task over the API rather than through the renderer. It never rendered a conditional and never submitted an optional field, so neither this defect nor the `fundingLimitations` type mismatch below could appear in a log. Both were found by a person using Tasklist, which is the only test that exercises the form as a form.